Aula 14 – Matrizes em C: Introdução

1. Objetivos de aprendizagem

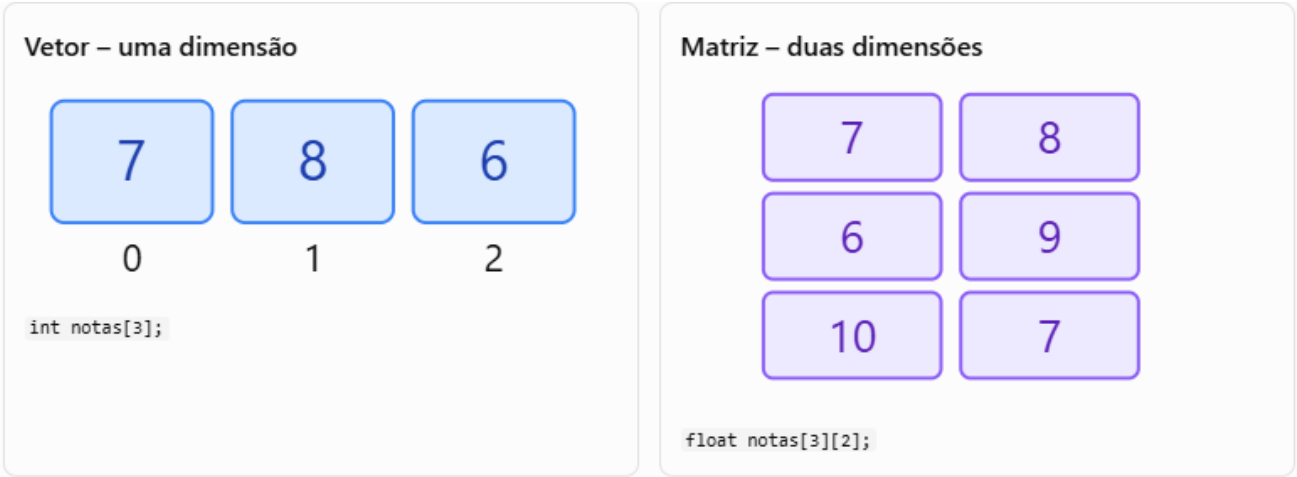
Objetivo geral: compreender o conceito de matriz como estrutura bidimensional, identificando sua organização em linhas e colunas e aplicando os conhecimentos de vetores e estruturas de repetição na linguagem C.

1.1 Objetivos específicos:

- Diferenciar vetores e matrizes.
- Identificar linhas, colunas e índices.
- Declarar, inicializar e acessar elementos de uma matriz.
- Utilizar for para percorrer matrizes.
- Resolver problemas simples envolvendo matrizes.

2. Relembrando... O que aprendemos sobre vetores?

Na aula anterior, aprendemos a armazenar várias notas em um vetor. Mas imagine que precisamos armazenar as notas de três alunos, sendo que cada aluno realizou duas provas. Como podemos organizar essas informações?



O vetor organiza elementos em uma dimensão, enquanto a matriz permite organizar dados em duas dimensões: linhas e colunas.

3. Situação-problema: como armazenar notas de vários alunos em diferentes provas?

Imagine que uma professora precisa armazenar as notas de três alunos em duas avaliações. Se utilizarmos variáveis individuais, teremos:

```
float nota1_aluno1, nota2_aluno1;
float nota1_aluno2, nota2_aluno2;
float nota1_aluno3, nota2_aluno3;
```

Funciona, mas imagine uma turma com 40 alunos e quatro avaliações. Seriam necessárias 160 variáveis!

Como podemos organizar essas informações de maneira mais simples?

3.1 O que é uma matriz em C?

Uma matriz é uma estrutura de dados que permite armazenar vários valores do mesmo tipo, organizados em linhas e colunas.

Diferentemente do vetor, que possui apenas uma dimensão, a matriz possui duas dimensões: linha e coluna.

3.2 Organizando os dados em uma tabela

Notas dos alunos

Aluno	Prova 1	Prova 2
Aluno 1	7,0	8,0
Aluno 2	6,0	9,0
Aluno 3	10,0	7,0

São 3 alunos × 2 provas = 6 notas.

Essa organização possui duas dimensões:

- Linhas: representam os alunos.
- Colunas: representam as avaliações.

Em C, podemos representar essa tabela utilizando uma matriz:

```
float notas[3][2] = {
   {7.0, 8.0},
   {6.0, 9.0},
   {10.0, 7.0}
};
```

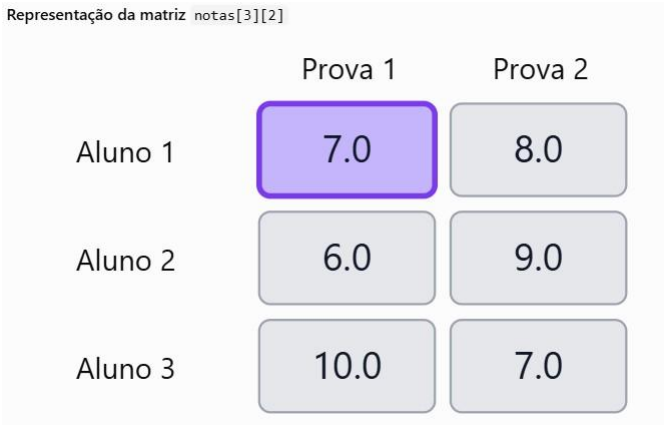
3.3 Mas não poderíamos utilizar um vetor?

Sim! Um vetor poderia armazenar as seis notas:

float notas[6] = {7, 8, 6, 9, 10, 7};

Entretanto, precisaríamos estabelecer uma regra para identificar quais posições pertencem a cada aluno.

Com uma matriz, essa organização fica explícita:



Por exemplo:

printf("%.1f", notas[1][0]);

Exibe 6.0, porque acessa a linha de índice 1 (segundo aluno) e a coluna de índice 0 (primeira prova).

3.4 Quais são as vantagens de utilizar matrizes?

1. Organização: representam naturalmente informações estruturadas em linhas e colunas.
2. Facilidade de acesso: permitem localizar elementos utilizando dois índices.
3. Processamento: facilitam cálculos por linha, por coluna ou envolvendo todos os elementos.
4. Menos repetição de código: podem ser percorridas utilizando laços de repetição aninhados.

Além das notas dos alunos, podemos utilizar matrizes para representar tabuleiros, mapas em grade, imagens digitais e tabelas de vendas.

Na aula anterior, aprendemos que um vetor permite armazenar vários valores do mesmo tipo utilizando uma única dimensão.

Mas existem problemas em que precisamos organizar informações em linhas e colunas, como uma tabela de notas, um tabuleiro ou um controle de vendas.

Para essas situações, podemos utilizar matrizes. Elas permitem armazenar e acessar os dados de maneira organizada, utilizando dois índices: um para a linha e outro para a coluna.

4. Sintaxe

Uma matriz é uma estrutura de dados que armazena elementos do mesmo tipo, organizados em linhas e colunas.

```
int numeros[3][2];
```

Nesse exemplo, temos três linhas e duas colunas, totalizando seis elementos. A posição de cada elemento é identificada por dois índices:

```
numeros[linha][coluna]
```

Em C, os índices começam em zero. Portanto, uma matriz numeros[3][2] possui linhas de 0 a 2 e colunas de 0 a 1.

4.1 Primeiro exemplo: matriz com valores definidos

```
#include <stdio.h>

int main(void) {
   int numeros[2][2] = {
      {10, 20},
      {30, 40}
   };

   printf("%d\n", numeros[0][0]);
   printf("%d\n", numeros[1][1]);

   return 0;
}
```

4.2 Percorrendo uma matriz com for

```
#include <stdio.h>

int main(void) {
   int numeros[2][2] = {
      {10, 20},
      {30, 40}
   };

   for (int i = 0; i < 2; i++) {
```

```
        for (int j = 0; j < 2; j++) {
            printf("%d ", numeros[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Exercicio de fixação – Revisão de conteúdo

Responda:

- Qual é a diferença entre vetor e matriz?
- Quantos elementos possui uma matriz int dados[3][4]?
- Por que normalmente utilizamos dois comandos for para percorrer uma matriz bidimensional?

Questão – Aplicação de Matrizes em C: Reserva de Assentos no Cinema

Um cinema possui 5 fileiras, com 8 assentos em cada uma. Para controlar as reservas, o sistema utiliza uma matriz, na qual:

- 0 representa um assento disponível.
- 1 representa um assento ocupado.

Considerando essa situação, qual alternativa apresenta corretamente a declaração da matriz em C e a forma de acessar o terceiro assento da segunda fileira?

a) ( ) int assentos[8][5]; e assentos[2][3]

b) ( ) int assentos[5][8]; e assentos[2][3]

c) ( ) int assentos[5][8]; e assentos[1][2]

d) ( ) int assentos[5]; e assentos[1][2]

Atividade para entrega:

1. Criar uma matriz 2×2 com valores definidos no código e exibi-la em formato de tabela.
2. Ler quatro valores digitados pelo usuário, armazená-los em uma matriz 2×2 e exibir a matriz.
3. Criar uma matriz 3×3, somar todos os seus elementos e apresentar o resultado.